

Enhanced Report System Integration Guide

Quick Start (10 Minutes)

Step 1: Install Dependencies

```
bash

pip install reportlab XlsxWriter
```

Step 2: Add Backend Files

```
bash

# Copy report routes to your routes directory
cp report_routes.py app/routes/

# Copy report service to your services directory
cp report_service.py app/services/
```

Step 3: Update init.py

```
python

# File: app/__init__.py

def create_app(config_name='development'):
    app = Flask(__name__)

    # ... existing configuration ...

    # Import blueprints
    from app.routes.report_routes import report_bp

    # Register blueprints
    app.register_blueprint(report_bp)

    # ... rest of configuration ...

    return app
```

Step 4: Replace Report Template

```
bash
```

Step 5: Verify Database Models

Ensure these models exist in your project:

-  `TrafficLightLog` (traffic_light.py)
-  `AIDecisionLog` (ai.py)
-  `EmergencyVehicleLog` (emergency_veh.py)
-  `TrafficPattern` (traffic_light.py)
-  `EmergencySchedule` (emergency_veh.py)
-  `GreenWaveSchedule` (emergency_veh.py)

Step 6: Create Route to Report Page

```
python

# In your main routes file (e.g., app/routes/main_routes.py)

@main_bp.route('/report')
def report_page():
    """Traffic report page"""
    return render_template('report.html')
```

Step 7: Restart Server

```
bash

python run.py
```

Step 8: Test

Navigate to: `http://localhost:5000/report`

Verification Tests

Test 1: Check API Endpoints

```
bash
```

```
# Test traffic overview
```

```
curl "http://localhost:5000/api/reports/traffic-overview?scenario=accra&date_start=2025-06-27&date_end=2025-07-04"
```

```
# Expected: JSON with traffic data
```

Test 2: Test Report Generation (PDF)

```
bash
```

```
curl -X POST http://localhost:5000/api/reports/generate \
-H "Content-Type: application/json" \
-d '{
  "template": "comprehensive",
  "date_start": "2025-06-27",
  "date_end": "2025-07-04",
  "format": "PDF",
  "metrics": ["Traffic Volume", "Average Speed"],
  "scenario": "accra"
}' \
--output test_report.pdf
```

Test 3: Test Emergency Data

```
bash
```

```
curl "http://localhost:5000/api/reports/emergency-activity?scenario=accra&date_start=2025-06-27&date_end=2025-07-04"
```

Test 4: Test Traffic Patterns

```
bash
```

```
curl "http://localhost:5000/api/reports/traffic-patterns?scenario=accra&date_start=2025-06-27&date_end=2025-07-04"
```



Key Features

1. Multi-Scenario Support

- Reports can be filtered by scenario (Accra, Kumasi, etc.)
- Each scenario maintains independent data
- Easy scenario switching in UI

2. Real File Generation

- **PDF Reports:** Professional formatted reports with tables and charts

- **Excel Reports:** Multi-sheet workbooks with formatted data
- **CSV Export:** Raw data export for further analysis

3. Emergency Vehicle Analytics

- Track ambulance, fire truck, and police responses
- Monitor green wave activations
- Analyze response times and priorities
- View scheduled vs detected emergencies

4. Traffic Pattern Integration

- Detect recurring traffic patterns
- Confidence scoring for patterns
- Traffic light-specific pattern analysis
- Actionable recommendations from patterns

5. Comprehensive Metrics

- Traffic volume by vehicle type
- Speed analysis over time
- Congestion heatmaps (day x hour)
- Peak hour detection
- AI performance metrics
- Traffic light performance grading

UI Enhancements

Modern Design Elements

- Gradient card headers
- Hover effects on cards
- Loading skeletons
- Toast notifications
- Responsive grid layouts
- Icon-enhanced sections

Interactive Features

- Real-time chart updates
- Filter persistence
- One-click exports
- Print optimization
- Date range validation

Report Templates

1. Comprehensive Report

Includes all available metrics and analysis.

2. Traffic Volume Summary

Focuses on vehicle counts and distribution.

3. Congestion Analysis

Deep dive into congestion patterns and heatmaps.

4. Emergency Response Report

Dedicated emergency vehicle analytics.

5. Traffic Pattern Analysis

Pattern detection and recommendations.

6. System Performance

AI optimization and traffic light performance.

Configuration

Scenario Names

Update scenario dropdown in `report.html`:

```
html

<select id="scenarioFilter" ...>
  <option value="accra">Accra Network</option>
  <option value="kumasi">Kumasi Network</option>
  <!-- Add your scenarios -->
</select>
```

Vehicle Type Distribution

Modify in `report_routes.py`:

```
python

distribution = {
    "Cars": round(total_vehicles * 0.68), # 68%
    "Trucks": round(total_vehicles * 0.15), # 15%
    "Buses": round(total_vehicles * 0.12), # 12%
    "Motorcycles": round(total_vehicles * 0.05) # 5%
}
```

PDF Styling

Customize colors in `generate_pdf_report()`:

```
python

title_style = ParagraphStyle(
    'CustomTitle',
    parent=styles['Heading1'],
    fontSize=24,
    textColor=colors.HexColor('#1e40af'), # Change color
    spaceAfter=30,
    alignment=TA_CENTER
)
```



Troubleshooting

Issue: "Module 'reportlab' not found"

Solution:

```
bash

pip install reportlab
```

Issue: "Module 'xlsxwriter' not found"

Solution:

```
bash

pip install XlsxWriter
```

Issue: "No emergency data showing"

Cause: EmergencyVehicleLog table is empty or not populated

Solution:

1. Verify emergency service is running
2. Check if emergency vehicles are being detected
3. Verify db_queue_service is storing data:

```
python  
  
# In your code  
print(f'Emergency logs count: {EmergencyVehicleLog.query.count()}')
```

Issue: "No traffic patterns displayed"

Cause: TrafficPattern table is empty

Solution:

1. Ensure traffic pattern detection is enabled
2. Check if sufficient data exists for pattern analysis
3. Verify pattern detection service is running

Issue: "PDF generation fails"

Check:

1. reportlab is installed: `pip list | grep reportlab`
2. Write permissions exist for temp files
3. Check error logs for specific reportlab errors

Issue: "Charts not rendering"

Solution:

1. Check browser console (F12)
2. Verify ECharts is loaded: `console.log(typeof echarts)`
3. Check if API endpoints return data
4. Verify chart containers have height: `#trafficVolumeChart { height: 350px; }`

Issue: "Report download fails"

Check:

1. Browser console for errors
2. Network tab shows successful response
3. File MIME type is correct
4. No popup blockers interfering

Database Schema Requirements

EmergencyVehicleLog

Required columns:

- `id` (String, PK)
- `scenario` (String)
- `vehicle_id` (String)
- `vehicle_type` (String)
- `priority` (String)
- `detected_at` (DateTime)
- `traffic_light_id` (String)
- `is_scheduled` (Boolean)

TrafficPattern

Required columns:

- `id` (String, PK)
- `scenario` (String)
- `traffic_light_id` (String)
- `pattern_type` (String)
- `confidence_score` (Float)
- `interval_start` (DateTime)
- `interval_end` (DateTime)
- `recommendations` (JSON)

GreenWaveSchedule

Required columns:

- `id` (String, PK)
- `emergency_id` (String, FK)
- `traffic_light_id` (String)
- `status` (String)
- `scheduled_start` (Float)



Performance Optimization

1. Database Indexes

```
sql

-- Add indexes for faster queries
CREATE INDEX idx_tl_logs_scenario_date
ON traffic_light_logs(scenario, created_at);

CREATE INDEX idx_emergency_scenario_date
ON emergency_vehicle_logs(scenario, detected_at);

CREATE INDEX idx_patterns_scenario_date
ON traffic_patterns(scenario, interval_start);
```

2. Caching

```
python

from flask_caching import Cache

cache = Cache(config={'CACHE_TYPE': 'simple'})

@cache.memoize(timeout=300) # 5 minutes
def get_traffic_overview(scenario, start, end):
    # Expensive query
    pass
```

3. Pagination

For large datasets, implement pagination:

```
python
```

```
page = request.args.get('page', 1, type=int)
per_page = request.args.get('per_page', 20, type=int)

logs = TrafficLightLog.query.paginate(
    page=page,
    per_page=per_page,
    error_out=False
)
```

Usage Examples

Generate PDF Report

```
javascript

// From frontend
const reportData = {
  template: 'comprehensive',
  date_start: '2025-06-27',
  date_end: '2025-07-04',
  format: 'PDF',
  metrics: [
    'Traffic Volume',
    'Average Speed',
    'Emergency Vehicles',
    'Traffic Patterns'
  ],
  scenario: 'accra'
};

const response = await fetch('/api/reports/generate', {
  method: 'POST',
  headers: { 'Content-Type': 'application/json' },
  body: JSON.stringify(reportData)
});

const blob = await response.blob();
const url = window.URL.createObjectURL(blob);
const a = document.createElement('a');
a.href = url;
a.download = 'traffic_report.pdf';
a.click();
```

Get Emergency Statistics

```
python
```

From backend

```
from app.models.emergency_veh import EmergencyVehicleLog
```

```
emergency_count = EmergencyVehicleLog.query.filter_by(  
    scenario='accra'  
)count()
```

```
by_type = db.session.query(  
    EmergencyVehicleLog.vehicle_type,  
    func.count(EmergencyVehicleLog.id)  
)filter_by(  
    scenario='accra'  
)group_by(EmergencyVehicleLog.vehicle_type).all()
```

Analyze Traffic Patterns

python

```
from app.models.traffic_light import TrafficPattern
```

```
high_confidence = TrafficPattern.query.filter(  
    TrafficPattern.scenario == 'accra',  
    TrafficPattern.confidence_score >= 80  
)order_by(TrafficPattern.confidence_score.desc()).all()
```

```
for pattern in high_confidence:  
    print(f'{pattern.traffic_light_id}: {pattern.pattern_type} ({pattern.confidence_score}%)')
```

Security Considerations

1. Input Validation

Always validate date inputs:

python

```
try:  
    start_date = datetime.strptime(date_start, '%Y-%m-%d')  
    end_date = datetime.strptime(date_end, '%Y-%m-%d')  
except ValueError:  
    return jsonify({"error": "Invalid date format"}), 400
```

2. Rate Limiting

Implement rate limiting for report generation:

python

```
from flask_limiter import Limiter

limiter = Limiter(app, key_func=get_remote_address)

@report_bp.route('/api/reports/generate')
@limiter.limit("10/hour")
def generate_report():
    # ...
```

3. File Size Limits

Limit report data range:

python

```
max_days = 90
if (end_date - start_date).days > max_days:
    return jsonify({
        "error": f'Date range exceeds maximum of {max_days} days'
    }), 400
```



Mobile Responsiveness

The report page is fully responsive with:

- Grid layouts adapt to screen size
- Touch-friendly controls
- Readable charts on small screens
- Optimized print layout

Test on different devices:

- Desktop: Full grid layout
- Tablet: 2-column layout
- Mobile: Single column, stacked cards



Next Steps

1. **Basic Integration** - Get reports working
2. **Test All Features** - Verify all endpoints
3. **Customize Design** - Adjust colors and styling

4.  **Add Authentication** - Protect report endpoints
5.  **Implement Caching** - Improve performance
6.  **Schedule Reports** - Auto-generate reports
7.  **Email Reports** - Send reports automatically
8.  **Dashboard Integration** - Link from main dashboard

Support

Common Questions

Q: Can I add custom metrics? A: Yes! Add to `_collect_report_data()` and update templates.

Q: How do I schedule automatic reports? A: Use a task scheduler like Celery or APScheduler.

Q: Can reports be emailed? A: Yes! Integrate with Flask-Mail and call generation endpoint.

Q: What's the maximum report size? A: Limited by available memory. Implement pagination for large datasets.

Q: Can I customize PDF layout? A: Yes! Modify `_generate_pdf_report()` function.

Success Checklist

- ☐ Dependencies installed
- ☐ Routes registered
- ☐ Report page accessible
- ☐ All API endpoints responding
- ☐ Emergency data showing
- ☐ Traffic patterns displaying
- ☐ PDF generation working
- ☐ Excel export working
- ☐ CSV export working
- ☐ Charts rendering properly
- ☐ Filters updating data
- ☐ Print layout working
- ☐ Mobile responsive
- ☐ No console errors

Once all checked, you're ready for production! 